# Spring Boot Interview Guide

Java 17 • Spring Boot 3.x • Detailed Reference with Code Examples

## 📋 Index


| # | Section | Topics Covered |
|---|---|---|
| 1 | Spring & Spring Boot | IoC, auto-configuration, starters, ApplicationContext, @SpringBootApplication |
| 2 | Request Internal Flow | DispatcherServlet lifecycle, Filter vs Interceptor vs AOP, @RequestBody |
| 3 | Annotations & Tricky Questions | Stereotype annotations, @Autowired types, @Configuration CGLIB, @Conditional, @Value |
| 4 | Bean Lifecycle & Scopes | Full lifecycle order, all scopes, BeanPostProcessor, prototype-in-singleton |
| 5 | Dependency Injection | Circular dependencies, 3-level cache, @Lazy, @Primary vs @Qualifier |
| 6 | JPA & Hibernate | Entity states, N+1 problem, L1/L2 cache, fetch types, GeneratedValue, save vs persist |
| 7 | Transactions | All 7 propagation types, isolation levels, rollback rules, TransactionTemplate |
| 8 | Spring Security | Filter chain, Auth vs Authz, JWT flow, @PreAuthorize, CSRF |
| 9 | Application Properties & Config | Property precedence, profiles, @ConfigurationProperties, key properties |
| 10 | Actuator & Monitoring | Key endpoints, custom HealthIndicator, Micrometer |
| 11 | AOP | Core concepts, JDK proxy vs CGLIB, @Around advice |
| 12 | Async & Scheduling | @Async internals, thread pool config, @Scheduled modes |
| 13 | Testing | @SpringBootTest vs slice tests, @MockBean vs @Mock, @SpyBean |
| 14 | Misc & Tricky | @ControllerAdvice, Spring events, caching annotations, CommandLineRunner, @Retryable |


## 1. Spring & Spring Boot

Core concepts, differences, and boot internals

## What is Spring Framework and what problems does it solve?

Easy

Before Spring, Java EE required heavy XML configuration, tight coupling, and container-managed EJBs. Spring introduced a lightweight alternative through two core principles:

- **IoC (Inversion of Control)** — The container controls object creation and lifecycle. You don't call `new MyService()`; Spring creates and injects it.
- **DI (Dependency Injection)** — Dependencies are "pushed in" by the container rather than the object fetching them.

### WITHOUT SPRING (TIGHT COUPLING)

```
public class OrderService {
    private PaymentService paymentService = new PaymentService(); // hard-coded, un
    private EmailService   emailService   = new EmailService();
}
```

### WITH SPRING (LOOSE COUPLING)

```
@Service
public class OrderService {
    private final PaymentService paymentService; // injected by Spring
    private final EmailService   emailService;

    public OrderService(PaymentService paymentService, EmailService emailService) {
        this.paymentService = paymentService;
        this.emailService   = emailService;
    }
}
```

Other pillars: **AOP** (cross-cutting concerns like logging, transactions), **Spring MVC** (web layer), **Spring Data** (data access abstraction), **Spring Security** (auth/authz).

## Spring vs Spring Boot — key differences?

Med

| Spring | Spring Boot |
|---|---|
| Manual XML/Java configuration | Auto-configuration via `@EnableAutoConfiguration` |
| No embedded server — deploy WAR to Tomcat | Embedded Tomcat/Jetty/Undertow — run as JAR |
| Manage dependency versions manually | Starter POMs + BOM manage all versions |
| No production features built-in | Actuator, health checks, metrics out-of-the-box |
| Hundreds of lines of boilerplate config | Convention over configuration — sensible defaults |

### SPRING BOOT MAIN CLASS — MINIMAL SETUP

```java
@SpringBootApplication  // replaces 100+ lines of XML config
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

> Spring Boot does NOT replace Spring — it's built on top of Spring and simplifies its setup.

## How does Spring Boot Auto-Configuration work internally?

Hard

@SpringBootApplication → @EnableAutoConfiguration → AutoConfigurationImportSelector → Reads AutoConfiguration.imports → Each class evaluated via @Conditional → Matching beans registered

Spring Boot 3.x reads from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (replaces old `spring.factories`). Each listed class is a `@Configuration` class guarded by conditions.

### EXAMPLE: DATASOURCEAUTOCONFIGURATION (SIMPLIFIED)

```java
@AutoConfiguration
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@ConditionalOnMissingBean(type = "io.r2dbc.spi.ConnectionFactory")
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean  // only if YOU haven't defined a DataSource bean
    public DataSource dataSource(DataSourceProperties props) {
        return props.initializeDataSourceBuilder().build();
    }
}
```

- `@ConditionalOnClass` — only registers if `DataSource` is on classpath
- `@ConditionalOnMissingBean` — your custom `DataSource` bean overrides this

> 💡 Run app with `--debug` or set `logging.level.org.springframework.boot.autoconfigure=DEBUG` to see CONDITIONS EVALUATION REPORT — shows every auto-config that matched or was skipped and why.

### What does @SpringBootApplication contain?

Tricky

It is a composed meta-annotation — shortcut for three annotations:

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration   // ← @Configuration
@EnableAutoConfiguration   // ← triggers auto-config
@ComponentScan             // ← scans current package + sub-packages
public @interface SpringBootApplication { ... }
```

#### CUSTOMISING SCAN BASE PACKAGES

```
// Option 1: attribute
@SpringBootApplication(scanBasePackages = { "com.myapp.service", "com.myapp.repo" }

// Option 2: exclude an auto-config
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
```

⚠️ If main class is in the **default package** (no package declaration), `@ComponentScan` scans all classes in the entire classpath — causing performance issues and unexpected bean registration. Always use a named package like `com.company.app`.

### BeanFactory vs ApplicationContext — differences?

Med

| Feature | BeanFactory | ApplicationContext |
|---|---|---|
| Bean instantiation | Lazy (on first getBean()) | Eager (all singletons at startup) |
| AOP support | No | Yes |
| Event publishing | No | Yes (ApplicationEventPublisher) |
| Internationalization | No | Yes (MessageSource) |
| Environment abstraction | No | Yes |
| Use case | Resource-constrained environments | All standard Spring apps |

Spring Boot uses `AnnotationConfigServletWebServerApplicationContext` for servlet web apps and `AnnotationConfigReactiveWebServerApplicationContext` for reactive apps.

```
// Getting beans programmatically
@Autowired
ApplicationContext ctx;

MyService svc = ctx.getBean(MyService.class);
String[]  names = ctx.getBeanDefinitionNames(); // all registered beans
```

## 2. Request Internal Flow

How an HTTP request travels through Spring MVC

## Walk through a Spring MVC request lifecycle in detail

Hard

1. HTTP Request → 2. Servlet Filters → 3. DispatcherServlet → 4. HandlerMapping → 5. Interceptor preHandle → 6. HandlerAdapter + ArgumentResolvers → 7. Controller Method → 8. ReturnValueHandler / MessageConverter → 9. Interceptor postHandle → 10. afterCompletion → 11. HTTP Response

### STEP-BY-STEP DETAIL

- **Filters (step 2)** — Servlet-level, run before Spring. Used for CORS, auth token extraction, request wrapping. Registered as `@Component` implementing `Filter` or via `FilterRegistrationBean`.
- **DispatcherServlet (step 3)** — Front Controller. Auto-registered by Spring Boot mapped to `/`. Delegates everything.
- **HandlerMapping (step 4)** — `RequestMappingHandlerMapping` scans all `@RequestMapping` methods and picks the best match based on URL, HTTP method, headers, params.
- **Interceptor preHandle (step 5)** — Returns `true` to continue or `false` to abort. Can set response directly.
- **HandlerAdapter (step 6)** — `RequestMappingHandlerAdapter` resolves method arguments (`@RequestBody`, `@PathVariable`, `@RequestParam`, `Principal`, etc.) using `HandlerMethodArgumentResolver` implementations.
- **MessageConverter (step 8)** — For REST, `MappingJackson2HttpMessageConverter` serialises return object to JSON.
- **afterCompletion (step 10)** — Always runs even if exception occurred. Good for resource cleanup.

### CUSTOM INTERCEPTOR EXAMPLE

```
@Component
public class RequestLoggingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Objec
        log.info("Incoming: {} {}", req.getMethod(), req.getRequestURI());
        req.setAttribute("startTime", System.currentTimeMillis());
        return true; // continue
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
                                Object handler, Exception ex) {
        long duration = System.currentTimeMillis() - (long) req.getAttribute("start
```

```java
        log.info("Completed in {}ms, status={}", duration, res.getStatus());
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Autowired RequestLoggingInterceptor interceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(interceptor).addPathPatterns("/api/**");
    }
}
```

## Filter vs Interceptor vs AOP Aspect — detailed comparison

Tricky

| Feature | Filter | Interceptor | AOP Aspect |
|---|---|---|---|
| Scope | Servlet container level | Spring MVC level | Any Spring bean method |
| Spring beans accessible | No (unless DelegatingFilterProxy) | Yes | Yes |
| Access to handler info | No | Yes (`HandlerMethod`) | Via JoinPoint |
| Applies to static resources | Yes | No | No |
| Granularity | Request/Response | Controller methods | Any method on any bean |
| Use case | CORS, encoding, security token extraction | Auth check, locale, logging | Transaction, caching, audit logging |

### FILTER — CORS EXAMPLE

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorsFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain
            throws IOException, ServletException {
        HttpServletResponse response = (HttpServletResponse) res;
        response.setHeader("Access-Control-Allow-Origin", "*");
        chain.doFilter(req, res); // must call this or request is blocked
    }
}
```

### How does @RequestBody deserialization work internally?

Med

1. DispatcherServlet passes to `RequestMappingHandlerAdapter`
2. It finds `RequestResponseBodyMethodProcessor` as the argument resolver for `@RequestBody`
3. Processor iterates registered `HttpMessageConverter`s — picks one whose `canRead()` returns true based on `Content-Type` header
4. `MappingJackson2HttpMessageConverter` uses Jackson `ObjectMapper` to deserialize JSON stream into the target type
5. If `@Valid` or `@Validated` present, `MethodValidationInterceptor` triggers Bean Validation (JSR-380)

```
@PostMapping("/orders")
public ResponseEntity<OrderResponse> createOrder(
        @RequestBody @Valid CreateOrderRequest request) { // ← deserialized + valid
    return ResponseEntity.status(HttpStatus.CREATED).body(orderService.create(reque
}

public record CreateOrderRequest(
    @NotBlank  String productId,
    @Min(1)    int quantity,
    @NotNull   BigDecimal price
) {}
```

⚠️ `@RequestBody` reads the servlet InputStream — it can only be read once. If you need to read it in a Filter too, wrap the request: `new ContentCachingRequestWrapper(request)`

## 3. Annotations & Tricky Questions

Internals and gotchas for the most commonly asked annotations

## @Component vs @Service vs @Repository vs @Controller

Tricky

All four are stereotype annotations — all trigger component scanning. They are specializations of `@Component` with added semantics:

| Annotation | Extra Behaviour | Layer |
|---|---|---|
| `@Component` | None — generic bean | Any |
| `@Repository` | Enables **persistence exception translation** — wraps DB-specific exceptions (SQLException, JPA exceptions) into Spring's `DataAccessException` hierarchy | Data access |
| `@Service` | None currently — reserved for future use by Spring. Acts as documentation. | Business logic |
| `@Controller` | Detected by `DispatcherServlet` as web handler. Methods can return view names. | Web (MVC) |
| `@RestController` | `@Controller` + `@ResponseBody` on all methods. Return values are serialized directly to HTTP response body. | Web (REST) |

### EXCEPTION TRANSLATION IN ACTION (@REPOSITORY)

```java
@Repository
public class UserRepository {
    public User findById(long id) {
        // If a DB-specific exception (e.g. PSQLException) is thrown here,
        // Spring translates it to DataAccessException automatically.
        // Without @Repository, raw vendor-specific exception propagates.
    }
}
```

## @Autowired — field vs constructor vs setter injection

Tricky

### FIELD INJECTION (AVOID)

```
@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService; // can't be final, can't test without Sp
}
```

### SETTER INJECTION (OPTIONAL DEPENDENCIES)

```
@Service
public class OrderService {
    private PaymentService paymentService;

    @Autowired(required = false) // optional dependency
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

### CONSTRUCTOR INJECTION ✅ RECOMMENDED

```
@Service
@RequiredArgsConstructor // Lombok — generates constructor for all final fields
public class OrderService {
    private final PaymentService paymentService; // immutable, clearly required
    private final EmailService   emailService;
    // Spring 4.3+: @Autowired not needed if single constructor
}
```

Why constructor injection wins: fields can be `final` (immutable), dependencies are explicit, easy to unit test without Spring container, no NullPointerException surprises.

### Multiple beans of same type — how to resolve ambiguity?

Tricky

```
public interface NotificationService { void send(String msg); }

@Service public class EmailNotificationService  implements NotificationService { ..
@Service public class SmsNotificationService    implements NotificationService { ..
@Service public class PushNotificationService   implements NotificationService { ..
```

#### OPTION 1: @PRIMARY — DEFAULT BEAN

```
@Primary @Service
public class EmailNotificationService implements NotificationService { ... }
```

#### OPTION 2: @QUALIFIER — EXPLICIT SELECTION

```
@Autowired
@Qualifier("smsNotificationService")
private NotificationService notificationService;
```

#### OPTION 3: INJECT ALL IMPLEMENTATIONS

```
@Autowired
private List<NotificationService> allNotifiers; // [email, sms, push]

@Autowired
private Map<String, NotificationService> notifierMap;
// {"emailNotificationService": ..., "smsNotificationService": ..., ...}

// Usage: send via all channels
public void broadcast(String msg) {
    allNotifiers.forEach(n -> n.send(msg));
}
```

## @Configuration vs @Component — CGLIB proxy difference

Hard

`@Configuration` classes are subclassed by CGLIB at runtime. Every `@Bean` method call is intercepted and returns the existing singleton rather than creating a new object.

### @CONFIGURATION — SINGLETON GUARANTEED

```
@Configuration
public class AppConfig {

    @Bean
    public ServiceA serviceA() {
        return new ServiceA(sharedRepo()); // sharedRepo() intercepted → returns si
    }

    @Bean
    public ServiceB serviceB() {
        return new ServiceB(sharedRepo()); // same instance as above!
    }

    @Bean
    public SharedRepo sharedRepo() { return new SharedRepo(); }
}
```

### @COMPONENT (LITE MODE) — CREATES NEW INSTANCE EACH CALL!

```
@Component  // NOT proxied by CGLIB
public class AppConfig {
    @Bean public ServiceA serviceA() { return new ServiceA(sharedRepo()); }
    @Bean public ServiceB serviceB() { return new ServiceB(sharedRepo()); }
    // BUG: serviceA and serviceB each get a DIFFERENT SharedRepo instance!
    @Bean public SharedRepo sharedRepo() { return new SharedRepo(); }
}
```

💡 Use `@Configuration(proxyBeanMethods = false)` when `@Bean` methods never call each other — avoids CGLIB overhead and is faster at startup (Spring Boot internal configs use this).

## @Transactional on private method — does it work? Self-invocation trap?

Tricky

Spring AOP works by wrapping your bean in a proxy. The proxy intercepts method calls. Two cases where `@Transactional` is silently ignored:

### CASE 1: PRIVATE METHOD — PROXY CAN'T SEE IT

```
@Service
public class UserService {
    @Transactional  // ❌ IGNORED — CGLIB cannot override private methods
    private void saveUserInternal(User user) { ... }
}
```

### CASE 2: SELF-INVOCATION — BYPASSES PROXY

```
@Service
public class UserService {

    public void createUser(User user) {
        saveUserInternal(user); // ❌ direct call on 'this', NOT via proxy → no tra
    }

    @Transactional
    public void saveUserInternal(User user) { ... }
}
```

### FIX: INJECT SELF OR EXTRACT TO ANOTHER BEAN

```
@Service
public class UserService {
    @Autowired
    private UserService self; // inject proxy of itself

    public void createUser(User user) {
        self.saveUserInternal(user); // ✅ goes through proxy → transaction starts
    }

    @Transactional
    public void saveUserInternal(User user) { ... }
}
```

⚠️ Same proxy limitation applies to `@Async`, `@Cacheable`, `@Retryable`, and all AOP-based annotations.

## @Value vs @ConfigurationProperties — when to use which?

Med

### @VALUE — SINGLE PROPERTY, SUPPORTS SPEL

```java
@Service
public class PaymentService {
    @Value("${payment.gateway.url}")
    private String gatewayUrl;

    @Value("${payment.timeout:5000}")     // default value 5000 if not set
    private int timeoutMs;

    @Value("#{systemProperties['java.home']}") // SpEL expression
    private String javaHome;
}
```

### @CONFIGURATIONPROPERTIES — GROUPED, TYPE-SAFE, VALIDATED

```yaml
# application.yml
payment:
  gateway:
    url: https://pay.example.com
    timeout: 5000
    retry-count: 3
    supported-currencies:
      - USD
      - EUR
```

```java
@ConfigurationProperties(prefix = "payment.gateway")
@Validated
public class PaymentGatewayProperties {

    @NotBlank
    private String url;

    @Min(1000) @Max(30000)
    private int timeout;

    private int retryCount = 3; // default value

    private List<String> supportedCurrencies;
```

```
    // getters/setters or use record
}
```

### @PostConstruct vs InitializingBean vs @Bean(initMethod) — order and use

Med

```
@Component
public class CacheManager implements InitializingBean {

    @Autowired
    private DataSource dataSource; // step 1: injected

    @PostConstruct
    public void init() {
        // step 2: @PostConstruct runs first — good for validation/logging
        log.info("DataSource injected: {}", dataSource != null);
        warmUpCache();
    }

    @Override
    public void afterPropertiesSet() {
        // step 3: InitializingBean — runs after @PostConstruct
        validateConfiguration();
    }

    @PreDestroy
    public void cleanup() {
        // called on context shutdown — clear cache, close connections
        cache.clear();
    }
}
```

**Execution order:** Constructor → `@Autowired` field injection → `@PostConstruct` → `afterPropertiesSet()` → `@Bean(initMethod)`

> ✅ Prefer `@PostConstruct` — it's JSR-250 standard (framework-agnostic), no Spring API coupling. Use `@Bean(initMethod)` when you can't modify the class (third-party libraries).

## 4. Bean Lifecycle & Scopes

How Spring creates, initialises, and destroys beans

## Explain the complete Spring Bean Lifecycle

Hard

1. Instantiation (Constructor) → 2. Populate Properties (@Autowired) →
3. BeanNameAware.setBeanName() → 4. BeanFactoryAware.setBeanFactory() →
5. ApplicationContextAware.setApplicationContext() → 6. BPP.postProcessBeforeInit() →
7. @PostConstruct → 8. InitializingBean.afterPropertiesSet() → 9. @Bean(initMethod) →
10. BPP.postProcessAfterInit() ← AOP proxy created here → 11. Bean Ready →
12. @PreDestroy → 13. DisposableBean.destroy()

### OBSERVING LIFECYCLE WITH A DEMO BEAN

```java
@Component
public class LifecycleDemo implements
        BeanNameAware, BeanFactoryAware, ApplicationContextAware, InitializingBean,

    public LifecycleDemo() { log.info("1. Constructor called"); }

    @Autowired
    public void setDep(SomeDep dep) { log.info("2. @Autowired set"); }

    @Override public void setBeanName(String name) { log.info("3. BeanNameAware: {}

    @PostConstruct
    public void postConstruct() { log.info("7. @PostConstruct"); }

    @Override public void afterPropertiesSet() { log.info("8. afterPropertiesSet");

    @PreDestroy
    public void preDestroy() { log.info("12. @PreDestroy"); }

    @Override public void destroy() { log.info("13. DisposableBean.destroy"); }
}
```

## Bean Scopes — all types with examples

Med

| Scope | Instances | Lifecycle |
|---|---|---|
| `singleton` | 1 per ApplicationContext | Created at startup, destroyed at shutdown |
| `prototype` | New per injection/getBean() | Created on demand, Spring does NOT destroy |
| `request` | 1 per HTTP request | Created on request arrival, destroyed when response committed |
| `session` | 1 per HTTP session | Tied to HttpSession lifecycle |
| `application` | 1 per ServletContext | Like singleton but per web app context |

### PROTOTYPE BEAN IN A SINGLETON — THE BUG

```
@Component
@Scope("prototype")
public class TaskProcessor { // stateful — needs new instance each use
    private List<String> results = new ArrayList<>();
}

@Service
public class JobService {
    @Autowired
    private TaskProcessor processor; // ❌ SAME prototype injected once — shared st
}
```

### FIX WITH OBJECTPROVIDER

```
@Service
public class JobService {
    @Autowired
    private ObjectProvider<TaskProcessor> processorProvider;

    public void runJob() {
        TaskProcessor processor = processorProvider.getObject(); // ✅ fresh instar
        processor.process();
    }
}
```

## 5. Dependency Injection

IoC, circular dependencies, lazy loading

### How does Spring resolve circular dependencies?

Hard

Spring uses a **3-level cache** in `DefaultSingletonBeanRegistry` to handle circular dependencies for singleton beans with setter/field injection:

- **Level 1 — singletonObjects**: Fully initialized, ready-to-use beans
- **Level 2 — earlySingletonObjects**: Early exposed (not fully initialized), still being wired
- **Level 3 — singletonFactories**: Factory lambdas that can produce early bean references

#### A → B → A RESOLUTION FLOW

```
// A depends on B, B depends on A
@Service public class ServiceA { @Autowired ServiceB b; }
@Service public class ServiceB { @Autowired ServiceA a; }

// Step 1: Create ServiceA instance — not fully initialized yet
// Step 2: Put ServiceA factory in level-3 cache
// Step 3: Start creating ServiceB (A's dependency)
// Step 4: ServiceB needs ServiceA → found in level-3 cache → early ref returned
// Step 5: ServiceB finishes initialization → moved to level-1 cache
// Step 6: ServiceA injects ServiceB → ServiceA fully initialized → level-1 cache
```

⚠️ **Constructor injection with circular dep always fails** — Spring can't create A without B, and B without A. Spring Boot 2.6+ also detects and blocks circular deps by default. Fix: break the cycle by extracting shared logic into a third bean, or use `@Lazy` on one side.

#### BREAKING CIRCULAR DEP WITH @LAZY

```
@Service
public class ServiceA {
    private final ServiceB b;
    public ServiceA(@Lazy ServiceB b) { // inject proxy, resolve B lazily
        this.b = b;
    }
}
```

## 6. JPA & Hibernate

## JPA Entity States — lifecycle with code

Hard

| State | In PersistenceContext? | DB-synced? |
|---|---|---|
| Transient | No | No |
| Managed (Persistent) | Yes | Yes — dirty checked at flush |
| Detached | No | Changes NOT tracked |
| Removed | Yes (briefly) | DELETE on flush |

```
EntityManager em = ...;

// TRANSIENT — no id, no persistence context
User user = new User("Alice");

// MANAGED — entity tracked. Any field change is auto-flushed to DB!
em.persist(user);
user.setEmail("alice@example.com"); // UPDATE will be issued automatically

// DETACHED — session closed, changes not tracked
em.detach(user); // or em.close() or end of @Transactional
user.setEmail("new@example.com"); // NOT synced to DB

// Re-attach with merge — returns a NEW managed copy
User managedUser = em.merge(user); // SELECT + UPDATE

// REMOVED
em.remove(managedUser); // DELETE on next flush
```

⚠️ **Dirty checking trap**: If you load an entity, modify it, and the method is `@Transactional` — Hibernate will issue an UPDATE automatically at flush time even without calling `save()`! This surprises many developers.

## N+1 problem — causes, detection, all fixes

Tricky

### THE PROBLEM

```
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)
    private Customer customer; // lazy — loaded on access
}

// BAD: 1 query for orders + N queries for each customer
List<Order> orders = orderRepo.findAll();  // SELECT * FROM orders (1 query)
orders.forEach(o -> System.out.println(o.getCustomer().getName())); // N queries!
```

### FIX 1: JPQL JOIN FETCH

```
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status")
List<Order> findByStatusWithCustomer(@Param("status") String status);
// 1 JOIN query → no lazy loads at all
```

### FIX 2: @ENTITYGRAPH

```
@EntityGraph(attributePaths = { "customer", "items" })
List<Order> findByStatus(String status); // Spring Data JPA — no JPQL needed
```

### FIX 3: @BATCHSIZE (REDUCES TO N/BATCHSIZE QUERIES)

```
@OneToMany(mappedBy = "order")
@BatchSize(size = 30) // Hibernate loads 30 collections in one IN clause
private List<OrderItem> items;
```

💡 Detect N+1 in tests with Hypersistence Optimizer or by enabling: `spring.jpa.properties.hibernate.generate_statistics=true` and checking query count in logs.

## L1 vs L2 Cache in Hibernate — detailed

Hard

### L1 CACHE — SESSION (ENTITYMANAGER) SCOPED

```
// L1 cache in action — within same transaction/session
User u1 = em.find(User.class, 1L); // SELECT FROM users WHERE id=1
User u2 = em.find(User.class, 1L); // NO SQL — returns from L1 cache
System.out.println(u1 == u2); // true — same object reference!
```

### L2 CACHE — SESSIONFACTORY SCOPED, SHARED ACROSS SESSIONS

```
# application.properties
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=\
  org.hibernate.cache.jcache.JCacheRegionFactory
spring.cache.jcache.provider=com.github.benmanes.caffeine.jcache.spi.CaffeineCachin
```

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE) // enable L2 for this entity
public class Product {
    @Id private Long id;
    private String name;
}

// Session 1
em1.find(Product.class, 1L); // DB hit → stored in L2 cache
// Session 2 (different transaction)
em2.find(Product.class, 1L); // L2 cache hit → no DB query!
```

| | L1 | L2 |
|---|---|---|
| Scope | Single EntityManager | Entire application (SessionFactory) |
| Default | Always on | Off — must configure |
| Eviction | Session close / `em.clear()` | TTL, explicit eviction, or on entity update |
| Provider | Hibernate built-in | Ehcache, Caffeine, Redis via JCache |

### save() vs saveAndFlush() vs persist() vs merge()

Tricky

```
// 1. persist() — JPA. Only for NEW (transient) entities. Cannot handle detached.
em.persist(new User("Alice")); // INSERT queued until flush

// 2. merge() — JPA. Returns a NEW managed copy. Input object stays detached.
User detached = ...; detached.setName("Bob");
User managed = em.merge(detached); // SELECT + UPDATE; detached unchanged
// Always use 'managed' after merge, not 'detached'!

// 3. Spring Data JPA save() — smart: persist if new (no ID), merge if existing
userRepo.save(new User("Charlie")); // → persist
userRepo.save(existingUser);          // → merge (has ID)

// 4. saveAndFlush() — save + immediately write to DB (useful in same-tx tests)
User saved = userRepo.saveAndFlush(user); // DB write now, not at tx commit
```

💡 `isNew()` detection in Spring Data: entity has no ID → new → persist. Has ID → merge. Override with `Persistable<ID>` for custom logic.

## 7. Transactions

Propagation, isolation, rollback rules, tricky gotchas

## All @Transactional propagation types with examples

Hard

| Propagation | Behaviour |
|---|---|
| `REQUIRED` (default) | Join existing tx; create new if none exists |
| `REQUIRES_NEW` | Always create NEW tx; suspend existing tx for duration |
| `SUPPORTS` | Join if tx exists; run non-transactionally if none |
| `NOT_SUPPORTED` | Always run non-transactionally; suspend existing tx |
| `MANDATORY` | Must have active tx; throw `IllegalTransactionStateException` if none |
| `NEVER` | Must NOT have tx; throw exception if tx exists |
| `NESTED` | Runs in nested tx (savepoint); partial rollback possible; outer tx commits/rolls back independently |

### REQUIRES_NEW — AUDIT LOG THAT MUST PERSIST EVEN IF MAIN TX ROLLS BACK

```
@Service
public class OrderService {
    @Autowired AuditService auditService;

    @Transactional
    public void placeOrder(Order order) {
        saveOrder(order);
        auditService.log("ORDER_PLACED"); // separate tx — persists even if this ro
        if (paymentFailed) throw new PaymentException(); // rolls back order but NO
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String event) {
        auditRepo.save(new AuditLog(event)); // own tx — commits independently
    }
}
```

### NESTED — SAVEPOINT WITHIN OUTER TX

```java
@Transactional
public void processItems(List<Item> items) {
    for (Item item : items) {
        try {
            itemService.processOne(item); // NESTED — savepoint before each item
        } catch (Exception e) {
            // roll back only this item's savepoint, continue outer tx
        }
    }
} // outer tx commits all successfully processed items
```

### Transaction Isolation Levels — what each prevents

Hard

| Level | Dirty Read | Non-Repeatable Read | Phantom Read | Performance |
|---|---|---|---|---|
| `READ_UNCOMMITTED` | ❌ allowed | ❌ allowed | ❌ allowed | Fastest |
| `READ_COMMITTED` (PG default) | ✅ prevented | ❌ allowed | ❌ allowed | Fast |
| `REPEATABLE_READ` (MySQL default) | ✅ prevented | ✅ prevented | ❌ allowed | Medium |
| `SERIALIZABLE` | ✅ prevented | ✅ prevented | ✅ prevented | Slowest |

- **Dirty Read**: Tx A reads uncommitted data from Tx B. Tx B rolls back → A read garbage.
- **Non-Repeatable Read**: Tx A reads row twice; Tx B updates it between reads → different values.
- **Phantom Read**: Tx A runs same query twice; Tx B inserts row matching query → different row counts.

```java
@Transactional(isolation = Isolation.REPEATABLE_READ)
public BigDecimal calculateBalance(long accountId) {
    BigDecimal credits = creditRepo.sumByAccount(accountId);
    BigDecimal debits  = debitRepo.sumByAccount(accountId);
    // Without REPEATABLE_READ, another tx could insert a row between queries
    return credits.subtract(debits);
}
```

**@Transactional rollback rules — all tricky cases** Tricky

### DEFAULT: ROLLS BACK ON RUNTIMEEXCEPTION AND ERROR ONLY

```
@Transactional
public void doWork() throws IOException {
    repo.save(data);
    throw new IOException("file not found"); // ❌ Checked exception — tx COMMITS!
}

@Transactional(rollbackFor = Exception.class) // ✅ rollback on any exception
public void doWorkSafe() throws IOException { ... }

@Transactional(noRollbackFor = StockNotFoundException.class) // don't rollback for
public void fulfillOrder() { ... }
```

### TRAP: SWALLOWED EXCEPTION → NO ROLLBACK

```
@Transactional
public void processPayment() {
    try {
        chargeCard();
    } catch (PaymentException e) {
        log.error("payment failed", e);
        // ❌ Exception swallowed — Spring doesn't know it failed → tx COMMITS!
    }
}

// Fix: mark tx as rollback-only manually if not rethrowing
@Transactional
public void processPaymentFixed(TransactionStatus status) {
    try {
        chargeCard();
    } catch (PaymentException e) {
        log.error("payment failed", e);
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly(); // ✅
    }
}
```

## 8. Spring Security

Authentication, authorization, filter chain, JWT

## Spring Security Filter Chain — detailed flow

Hard

Request → DelegatingFilterProxy → FilterChainProxy →
Matches SecurityFilterChain by URL → SecurityContextPersistenceFilter →
UsernamePasswordAuthFilter or BearerTokenFilter → ExceptionTranslationFilter →
AuthorizationFilter → Controller

### MODERN SPRING SECURITY 6 CONFIGURATION

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)      // stateless JWT API
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.cl
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customEntryPoint)
                .accessDeniedHandler(customAccessDeniedHandler))
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // strength 12 — ~250ms per hash
    }
}
```

## JWT Authentication — full implementation

Hard

### JWT FILTER

```
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain)
            throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(request, response); return;
        }

        String token    = header.substring(7);
        String username = jwtUtil.extractUsername(token);

        if (username != null && SecurityContextHolder.getContext().getAuthenticatio
            UserDetails userDetails = userDetailsService.loadUserByUsername(usernam
            if (jwtUtil.isValid(token, userDetails)) {
                var auth = new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(r
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(request, response);
    }
}
```

### JWT UTILITY

```
@Component
public class JwtUtil {
    @Value("${jwt.secret}") private String secret;
    @Value("${jwt.expiry:86400000}") private long expiryMs; // 24h default

    public String generateToken(UserDetails userDetails) {
        return Jwts.builder()
```

```
            .subject(userDetails.getUsername())
            .claim("roles", userDetails.getAuthorities())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiryMs))
            .signWith(Keys.hmacShaKeyFor(secret.getBytes()))
            .compact();
    }

    public boolean isValid(String token, UserDetails userDetails) {
        return extractUsername(token).equals(userDetails.getUsername())
            && !isExpired(token);
    }
}
```

### @PreAuthorize vs @Secured vs @RolesAllowed — with SpEL examples

Med

```java
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    // @PreAuthorize — most powerful, full SpEL support
    @GetMapping
    @PreAuthorize("hasRole('USER') or hasRole('ADMIN')")
    public List<Order> getOrders() { ... }

    // Access method parameters in SpEL
    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN') or @orderSecurity.isOwner(#id, authentication)"
    public Order update(@PathVariable long id, @RequestBody Order order) { ... }

    // @PostAuthorize — check AFTER method runs (can inspect return value)
    @GetMapping("/{id}")
    @PostAuthorize("returnObject.userId == authentication.principal.id")
    public Order getById(@PathVariable long id) { ... }

    // @Secured — simple, no SpEL, requires ROLE_ prefix
    @DeleteMapping("/{id}")
    @Secured("ROLE_ADMIN")
    public void delete(@PathVariable long id) { ... }
}
```

Enable with `@EnableMethodSecurity` (Spring Security 6+). This replaces deprecated `@EnableGlobalMethodSecurity`.

## 9. Application Properties & Configuration

Configuration files, profiles, externalized config

### Property source precedence — all 12 levels

Hard

Spring Boot evaluates properties in this order (higher = wins). Lower-numbered = higher priority:

1. Command-line arguments: `--server.port=9090`
2. `SPRING_APPLICATION_JSON` (env var with embedded JSON)
3. Servlet config init params (`web.xml`)
4. Servlet context init params
5. JNDI attributes from `java:comp/env`
6. Java system properties (`-Dserver.port=9090`)
7. OS environment variables (`SERVER_PORT=9090`)
8. Profile-specific config **outside** jar: `application-prod.yml`
9. Config files **outside** jar: `application.yml`
10. Profile-specific config **inside** jar: `application-prod.yml`
11. Config files **inside** jar: `application.yml`
12. `@PropertySource` annotations on `@Configuration` classes

> 💡 OS env variables use relaxed binding: `SERVER_PORT` maps to `server.port`, `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`. This is how Kubernetes/Docker secrets work.

## Spring Profiles — complete guide

Med

### PROFILE-SPECIFIC FILES

```yaml
# application.yml — base config (always loaded)
spring:
  application:
    name: my-service
server:
  port: 8080

---
# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:devdb
  jpa:
    hibernate.ddl-auto: create-drop
logging.level.root: DEBUG

---
# application-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST}:5432/${DB_NAME}
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate.ddl-auto: validate
logging.level.root: WARN
```

### PROFILE-CONDITIONAL BEANS

```java
@Configuration
public class StorageConfig {

    @Bean
    @Profile("dev")
    public StorageService localStorageService() {
        return new LocalFileStorageService("/tmp/uploads");
    }

    @Bean
    @Profile("prod")
    public StorageService s3StorageService() {
```

```
        return new S3StorageService(awsConfig);
    }
}
```

### ACTIVATION METHODS

```
# In application.properties
spring.profiles.active=prod

# Command line
java -jar app.jar --spring.profiles.active=prod

# Environment variable (Docker/K8s)
SPRING_PROFILES_ACTIVE=prod

# Profile groups (activate multiple at once)
spring.profiles.group.production=prod,metrics,featureflags
```

### @ConfigurationProperties — full example with validation

Med

```yaml
# application.yml
app:
  email:
    host: smtp.gmail.com
    port: 587
    username: ${SMTP_USER}
    password: ${SMTP_PASS}
    retry-attempts: 3
    retry-delay: PT5S          # ISO-8601 Duration
    allowed-domains:
      - example.com
      - company.org
```

```java
@ConfigurationProperties(prefix = "app.email")
@Validated
@Component
public class EmailProperties {

    @NotBlank
    private String host;

    @Min(1) @Max(65535)
    private int port;

    @NotBlank
    private String username;

    private String password;

    @Min(1)
    private int retryAttempts = 3;

    private Duration retryDelay = Duration.ofSeconds(5); // auto-converted from PT5

    @NotEmpty
    private List<String> allowedDomains;

    // getters/setters or use @Data from Lombok
}
```

# 10. Actuator & Monitoring

Production-ready endpoints and health monitoring

### Spring Boot Actuator — endpoints and security

Med

| Endpoint | Description | Sensitive? |
|---|---|---|
| `/actuator/health` | Aggregated health: DB, disk, custom checks | Partial |
| `/actuator/metrics/{name}` | Micrometer metrics (JVM, HTTP, custom) | Yes |
| `/actuator/env` | All resolved properties (masks passwords) | Yes |
| `/actuator/beans` | All Spring beans + their dependencies | Yes |
| `/actuator/mappings` | All URL-to-handler mappings | Yes |
| `/actuator/loggers` | View and change log level at runtime | Yes |
| `/actuator/threaddump` | Current thread dump | Yes |
| `/actuator/heapdump` | Download heap dump | Yes |

```
# Only expose health and info publicly, secure rest
management.endpoints.web.exposure.include=health,info,metrics,loggers
management.endpoints.web.exposure.exclude=heapdump,threaddump
management.endpoint.health.show-details=when-authorized
management.server.port=8081  # separate port for actuator — don't expose to interne
```

#### CHANGE LOG LEVEL AT RUNTIME (NO RESTART!)

```
# POST /actuator/loggers/com.myapp.service
{ "configuredLevel": "DEBUG" }
```

## Custom HealthIndicator and custom Metric

Med

### CUSTOM HEALTHINDICATOR

```
@Component
public class ExternalApiHealth implements HealthIndicator {
    private final RestTemplate restTemplate;

    @Override
    public Health health() {
        try {
            ResponseEntity<String> resp = restTemplate.getForEntity("https://api.pa
            if (resp.getStatusCode().is2xxSuccessful())
                return Health.up().withDetail("latency", "ok").build();
            return Health.down().withDetail("status", resp.getStatusCode()).build()
        } catch (Exception e) {
            return Health.down(e).withDetail("error", e.getMessage()).build();
        }
    }
}
// Exposed at: /actuator/health/externalApi
```

### CUSTOM MICROMETER METRIC

```
@Service
public class OrderService {
    private final Counter orderCounter;
    private final Timer   processingTimer;

    public OrderService(MeterRegistry registry) {
        orderCounter    = registry.counter("orders.created", "app", "my-service");
        processingTimer = registry.timer("orders.processing.duration");
    }

    public Order createOrder(OrderRequest req) {
        return processingTimer.record(() -> {
            orderCounter.increment();
            return doCreate(req);
        });
    }
}
```

# 11. AOP — Aspect Oriented Programming

Proxies, pointcuts, advice types

## AOP core concepts with a full working example

Med

| Term | Meaning |
|---|---|
| Aspect | Module that encapsulates cross-cutting concern (`@Aspect` class) |
| Advice | Code to run: `@Before`, `@After`, `@Around`, `@AfterReturning`, `@AfterThrowing` |
| Join Point | Point in execution — in Spring AOP, always a method execution |
| Pointcut | Expression selecting which join points to advise |
| Target Object | The original bean being advised (wrapped in proxy) |
| Weaving | Applying aspects — Spring does runtime proxy weaving |

```java
@Aspect
@Component
public class AuditAspect {

    // Pointcut: all methods in service package
    @Pointcut("execution(* com.myapp.service.*.*(..))")
    public void serviceLayer() {}

    // Before — runs before method
    @Before("serviceLayer()")
    public void logEntry(JoinPoint jp) {
        log.info("Calling {} with args {}", jp.getSignature().getName(), jp.getArgs
    }

    // AfterReturning — inspect return value
    @AfterReturning(pointcut = "serviceLayer()", returning = "result")
    public void logResult(JoinPoint jp, Object result) {
        log.info("{} returned {}", jp.getSignature().getName(), result);
    }

    // AfterThrowing — inspect exception
    @AfterThrowing(pointcut = "serviceLayer()", throwing = "ex")
    public void logError(JoinPoint jp, Exception ex) {
        log.error("{} threw {}", jp.getSignature().getName(), ex.getMessage());
    }

    // Around — most powerful, controls execution
```

```java
    @Around("@annotation(com.myapp.annotation.Retry)")
    public Object retryOnFailure(ProceedingJoinPoint pjp) throws Throwable {
        int attempts = 0;
        while (true) {
            try {
                return pjp.proceed(); // invoke actual method
            } catch (Exception e) {
                if (++attempts >= 3) throw e;
                Thread.sleep(1000L * attempts);
            }
        }
    }
}
```

### JDK Dynamic Proxy vs CGLIB — detailed difference

Hard

#### JDK DYNAMIC PROXY

```
// Works ONLY if target implements an interface
public interface PaymentService { void pay(); }

@Service
public class PaymentServiceImpl implements PaymentService { ... }

// Spring creates: Proxy.newProxyInstance(PaymentService.class, handler)
// Injecting as interface → JDK proxy used
// ✅ @Autowired PaymentService payment;
// ❌ @Autowired PaymentServiceImpl payment; — ClassCastException if JDK proxy
```

#### CGLIB PROXY

```
// Subclasses the class directly — no interface needed
@Service
public class OrderService { // no interface
    public void createOrder() { ... }
}
// Spring Boot default: proxyTargetClass=true → always uses CGLIB
// CGLIB creates: class OrderService$$SpringCGLIB extends OrderService { ... }

// ❌ CGLIB cannot proxy these:
public final class FinalService { ... }        // can't subclass final
public final void doSomething() { ... }         // can't override final method
```

⚠️ Never make Spring-managed beans or their important methods `final` — CGLIB proxy will fail silently or throw an error.

## 12. Async & Scheduling

@Async, @Scheduled, ThreadPoolTaskExecutor

## @Async — internals, thread pool config, CompletableFuture

Tricky

### PROPER THREAD POOL CONFIGURATION

```
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(5);          // always-alive threads
        exec.setMaxPoolSize(20);          // max threads under load
        exec.setQueueCapacity(100);       // tasks queued before new threads spawne
        exec.setKeepAliveSeconds(60);     // idle threads above core die after 60s
        exec.setThreadNamePrefix("async-");
        exec.setRejectedExecutionHandler(new CallerRunsPolicy()); // don't drop tas
        exec.initialize();
        return exec;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            log.error("Async error in {}: {}", method.getName(), ex.getMessage());
    }
}
```

### USAGE WITH COMPLETABLEFUTURE

```
@Service
public class ReportService {

    @Async
    public CompletableFuture<Report> generateReport(long userId) {
        // runs in thread pool, not caller thread
        Report report = expensiveCalculation(userId);
        return CompletableFuture.completedFuture(report);
    }
}

// Parallel async calls
CompletableFuture<Report> salesFuture  = reportService.generateReport(1);
CompletableFuture<Report> stockFuture  = reportService.generateReport(2);
```

```
CompletableFuture.allOf(salesFuture, stockFuture).join(); // wait for both
Report sales = salesFuture.get();
Report stock = stockFuture.get();
```

### @Scheduled — all modes with cron examples

Med

```
@Component
public class ScheduledTasks {

    // fixedRate: fires every 5s regardless of last execution duration
    @Scheduled(fixedRate = 5000)
    public void heartbeat() { log.info("ping"); }

    // fixedDelay: 5s AFTER previous execution finishes
    @Scheduled(fixedDelay = 5000, initialDelay = 10000) // start after 10s
    public void cleanupTemp() { deleteOldFiles(); }

    // Cron: "sec min hour day month weekday"
    @Scheduled(cron = "0 0 2 * * *")       // every day at 02:00
    public void nightly() { runNightlyBatch(); }

    @Scheduled(cron = "0 0 9 * * MON-FRI") // 9AM weekdays
    public void dailyReport() { sendReport(); }

    @Scheduled(cron = "0 0/30 8-18 * * MON-FRI") // every 30min, 8AM-6PM weekdays
    public void businessHoursCheck() { checkInventory(); }

    // From properties - flexible
    @Scheduled(cron = "${scheduler.sync.cron:0 0 * * * *}")
    public void sync() { syncData(); }
}
```

⚠️ Default scheduler is **single-threaded**. If one task is slow, others queue up. Fix: `spring.task.scheduling.pool.size=5`

## 13. Testing

Unit tests, slice tests, Mockito integration

## @SpringBootTest vs slice test annotations — when to use each

Med

### @WEBMVCTEST — CONTROLLER LAYER ONLY, NO DB

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mockMvc;
    @MockBean  OrderService orderService; // service mocked — not in web slice

    @Test
    void createOrder_returns201() throws Exception {
        var order = new Order(1L, "PENDING");
        given(orderService.create(any())).willReturn(order);

        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""{"productId":"P1","quantity":2}"""))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1));
    }
}
```

### @DATAJPATEST — JPA/REPOSITORY LAYER, H2 IN-MEMORY

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired UserRepository userRepo;
    @Autowired TestEntityManager em; // helper for test data setup

    @Test
    void findByEmail_returnsUser() {
        em.persistAndFlush(new User("alice@example.com"));
        Optional<User> found = userRepo.findByEmail("alice@example.com");
        assertThat(found).isPresent();
        assertThat(found.get().getEmail()).isEqualTo("alice@example.com");
    }
}
```

| Annotation | Context loaded | Speed |
|---|---|---|
| `@SpringBootTest` | Full context | Slow |

| | | |
|---|---|---|
| `@WebMvcTest` | Web layer (controllers, filters, advice) | Fast |
| `@DataJpaTest` | JPA layer + H2 | Fast |
| `@RestClientTest` | RestTemplate/WebClient | Fast |
| `No annotation (plain)` | No context — pure Mockito | Fastest |

## @MockBean vs @Mock vs @SpyBean — differences

Tricky

### @MOCK — PURE MOCKITO, NO SPRING CONTEXT

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock       PaymentService paymentService; // Mockito mock
    @InjectMocks OrderService   orderService;  // inject mocks into this

    @Test
    void placeOrder_callsPayment() {
        when(paymentService.charge(any())).thenReturn(true);
        orderService.placeOrder(new Order());
        verify(paymentService).charge(any());
    }
}
```

### @MOCKBEAN — REPLACES BEAN IN SPRING CONTEXT

```
@SpringBootTest
class OrderIntegrationTest {

    @Autowired  OrderService   orderService;
    @MockBean   PaymentService paymentService; // replaces real bean in context
    @MockBean   EmailService   emailService;

    @Test
    void fullFlowTest() {
        when(paymentService.charge(any())).thenReturn(true);
        var result = orderService.placeOrder(new Order());
        assertThat(result.getStatus()).isEqualTo("CONFIRMED");
    }
}
```

### @SPYBEAN — REAL BEAN + ABILITY TO OVERRIDE SPECIFIC METHODS

```
@SpringBootTest
class NotificationTest {

    @SpyBean EmailService emailService; // REAL bean, but spy on it

    @Test
    void sendEmail_calledOnce() {
```

```
        doNothing().when(emailService).sendEmail(any()); // stub specific method
        orderService.placeOrder(order);
        verify(emailService, times(1)).sendEmail(any());
    }
}
```

## 14. Misc & Tricky Questions

Exception handling, events, caching, and more

### @ControllerAdvice — global exception handling with problem details

Med

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handle custom business exception
    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex, WebRequest re
        return new ErrorResponse("NOT_FOUND", ex.getMessage(), req.getDescription(f
    }

    // Handle validation errors (@Valid failures)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, Object> handleValidation(MethodArgumentNotValidException ex)
        Map<String, String> errors = ex.getBindingResult().getFieldErrors().stream(
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage));
        return Map.of("status", 400, "errors", errors);
    }

    // Catch-all
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleAll(Exception ex) {
        log.error("Unhandled exception", ex);
        return new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred",
    }
}
```

## Spring Caching — @Cacheable, @CachePut, @CacheEvict in depth

Med

```
@Service
public class ProductService {

    // Cache result. Key = productId. On cache hit: method NOT called.
    @Cacheable(value = "products", key = "#productId")
    public Product findById(long productId) {
        log.info("DB hit for product {}", productId); // won't print on cache hit
        return productRepo.findById(productId).orElseThrow();
    }

    // Conditional caching — only cache active products
    @Cacheable(value = "products", key = "#id", condition = "#id > 0", unless = "#r
    public Product findConditional(long id) { ... }

    // Always update cache after save (write-through)
    @CachePut(value = "products", key = "#result.id")
    public Product save(Product product) {
        return productRepo.save(product); // method ALWAYS runs, cache ALWAYS updat
    }

    // Remove from cache on delete
    @CacheEvict(value = "products", key = "#productId")
    public void delete(long productId) { productRepo.deleteById(productId); }

    // Clear entire cache
    @CacheEvict(value = "products", allEntries = true)
    public void clearAll() {}

    // Multiple cache operations
    @Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "productList", allEntries = true)
    })
    public void deleteAndClearList(long id) { ... }
}
```

### REDIS CACHE CONFIG

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
```

```
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        var config = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()));
        return RedisCacheManager.builder(factory).cacheDefaults(config).build();
    }
}
```

## Spring Application Events — lifecycle and custom events

Med

### BUILT-IN STARTUP EVENTS (IN ORDER)

1. `ApplicationStartingEvent` — very early, logging/listeners only
2. `ApplicationEnvironmentPreparedEvent` — environment ready, not context yet
3. `ApplicationContextInitializedEvent` — context created, not refreshed
4. `ApplicationPreparedEvent` — beans definitions loaded, not instantiated
5. `ContextRefreshedEvent` — all beans instantiated
6. `ApplicationStartedEvent` — app started, before runners
7. `ApplicationReadyEvent` — ready to serve traffic (after CommandLineRunner/ApplicationRunner)
8. `ApplicationFailedEvent` — startup failed

### CUSTOM DOMAIN EVENTS

```
// Event class
public record OrderPlacedEvent(Order order, Instant timestamp) {}

// Publisher
@Service
@RequiredArgsConstructor
public class OrderService {
    private final ApplicationEventPublisher publisher;

    @Transactional
    public Order placeOrder(OrderRequest req) {
        Order order = orderRepo.save(new Order(req));
        publisher.publishEvent(new OrderPlacedEvent(order, Instant.now())); // sync
        return order;
    }
}

// Listener — same thread (transactional event)
@Component
public class OrderEventListener {

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onOrderPlaced(OrderPlacedEvent event) {
        // runs ONLY after transaction commits — prevents sending email if tx rolls
        emailService.sendOrderConfirmation(event.order());
    }

    @Async
```

```
    @EventListener
    public void updateAnalytics(OrderPlacedEvent event) {
        analyticsService.track(event); // async — doesn't block main thread
    }
}
```

💡 Use `@TransactionalEventListener` instead of `@EventListener` for events published inside `@Transactional` — ensures listener only fires after the transaction successfully commits.

### @Retryable — Spring Retry with backoff strategies

Med

```
@Configuration
@EnableRetry
public class RetryConfig {}

@Service
public class ExternalApiService {

    // Retry 3x with exponential backoff: 1s, 2s, 4s
    @Retryable(
        retryFor  = { HttpServerErrorException.class, ConnectTimeoutException.class
        maxAttempts = 3,
        backoff   = @Backoff(delay = 1000, multiplier = 2, maxDelay = 10000)
    )
    public ApiResponse callApi(String endpoint) {
        log.info("Attempting API call to {}", endpoint);
        return restTemplate.getForObject(endpoint, ApiResponse.class);
    }

    // Fallback — called after ALL retries exhausted
    @Recover
    public ApiResponse recoverApiCall(Exception ex, String endpoint) {
        log.error("All retries failed for {}: {}", endpoint, ex.getMessage());
        return ApiResponse.fallback(); // or throw, or return cached data
    }
}
```

⚠️ `@Recover` method must have the same return type and the first parameter must be the exception type. It also works via AOP proxy — same self-invocation limitation applies.

## CommandLineRunner vs ApplicationRunner vs @EventListener(ApplicationReadyEvent)

Easy

### COMMANDLINERUNNER — RAW ARGS AS STRING[]

```
@Component
@Order(1) // run first
public class DatabaseSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        if (args.length > 0 && args[0].equals("--seed")) seedDatabase();
    }
}
```

### APPLICATIONRUNNER — PARSED ARGS

```
@Component
@Order(2)
public class CacheWarmer implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        if (args.containsOption("warm-cache")) warmCache();
        // args.getNonOptionArgs(), args.getOptionValues("profile")
    }
}
```

### APPLICATIONREADYEVENT — FOR NON-STARTUP TASKS (NO ORDERING)

```
@EventListener(ApplicationReadyEvent.class)
public void onReady() {
    registerWithServiceDiscovery(); // register in Eureka/Consul after full startup
}
```

## Graceful shutdown — how it works and configuration

Med

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

### SHUTDOWN SEQUENCE

SIGTERM received → Stop accepting new requests (return 503) →
Wait for in-flight requests (up to 30s) → @PreDestroy / DisposableBean called →
Thread pools shut down → Context closed → JVM exits

```java
// Custom shutdown hook
@Component
public class ShutdownHandler {
    @PreDestroy
    public void onShutdown() {
        log.info("Application shutting down — flushing queues...");
        messageQueue.flush();
        cache.persist();
    }
}
```

> In Kubernetes: set `terminationGracePeriodSeconds` in Pod spec to match or exceed `timeout-per-shutdown-phase` so K8s doesn't SIGKILL before graceful shutdown completes.